

1. ការណែនាំអំពី PHP

PHP គឺជាភាសាសរសេរស្រ្តីបដែលត្រូវបានប្រើយ៉ាងទូលំទូលាយសម្រាប់ការអភិវឌ្ឍគេហទំព័រឌីណាមិក។ វាត្រូវបានប្រើសម្រាប់បង្កើតកម្មវិធីគេហទំព័រដែលមានភាពអន្តរកម្មខ្ពស់ និងអាចភ្ជាប់ជាមួយមូលដ្ឋានទិន្នន័យ។

1.1. តើ PHP គឺជាអ្វី?

PHP គឺជាភាសាសរសេរស្រ្តីបដែលត្រូវបានប្រើសម្រាប់ការអភិវឌ្ឍគេហទំព័រឌីណាមិក។ វាត្រូវបានបង្កើតឡើងដោយ Rasmus Lerdorf នៅឆ្នាំ 1994។ PHP មានន័យថា "PHP: Hypertext Preprocessor"។

```
<?php
    echo "Hello, World!";
?>
```

លទ្ធផល៖

```
Hello, World!
```

1.2. ប្រវត្តិនិងលក្ខណៈពិសេសនៃ PHP

PHP ត្រូវបានបង្កើតឡើងដោយ Rasmus Lerdorf នៅឆ្នាំ 1994។ វាត្រូវបានគេប្រើជាមធ្យោបាយសម្រាប់តាមដានការចូលមើលរបស់អ្នកប្រើប្រាស់នៅលើគេហទំព័រផ្ទាល់ខ្លួនរបស់គាត់។ ក្រោយមក PHP បានអភិវឌ្ឍជាភាសាសរសេរស្រ្តីបពេញលេញសម្រាប់ការអភិវឌ្ឍគេហទំព័រ។

លក្ខណៈពិសេសនៃ PHP:

- អាចប្រើប្រាស់ជាមួយ HTML បានយ៉ាងងាយស្រួល។
- គាំទ្រការភ្ជាប់មូលដ្ឋានទិន្នន័យជាច្រើន ដូចជា MySQL, PostgreSQL, MongoDB។
- មានសហគមន៍ធំ និងឯកសារដ៏សម្បូរបែប។

1.3. រក្សាសម្ព័ន្ធនិងរចនាសម្ព័ន្ធមូលដ្ឋាននៃ PHP

PHP មានរក្សាសម្ព័ន្ធដ៏សាមញ្ញ និងងាយស្រួលយល់។ កូដ PHP ត្រូវបានដាក់នៅក្នុងស្លាក `<?php ?>`។ ឧទាហរណ៍៖

```
<?php
    $name = "John";
    echo "Hello, " . $name;
?>
```

លទ្ធផល៖

Hello, John

1.4. ការដំណើរការ PHP លើម៉ាស៊ីនបម្រើ

PHP ត្រូវបានដំណើរការនៅលើម៉ាស៊ីនបម្រើ (Server)។ ដើម្បីដំណើរការ PHP អ្នកត្រូវការម៉ាស៊ីនបម្រើដែលគាំទ្រកូដ PHP ដូចជា Apache ឬ Nginx។ អ្នកក៏អាចប្រើកម្មវិធីដូចជា XAMPP ឬ WAMP ដើម្បីដំឡើងម៉ាស៊ីនបម្រើក្នុងស្ថានីយរបស់អ្នក។

ជំហានដើម្បីដំណើរការ PHP:

1. ដំឡើងម៉ាស៊ីនបម្រើដូចជា XAMPP ឬ WAMP។
2. បង្កើតឯកសារ PHP នៅក្នុងថតhtdocs (សម្រាប់ XAMPP)។
3. បើកឧបករណ៍រុករក និងវាយអាសយដ្ឋាន localhost/yourfile.php។

2. PHP Basics

PHP គឺជាភាសាស្រ្តីបដែលរត់នៅលើស៊ើវ ដែលត្រូវបានរចនាឡើងសម្រាប់ការអភិវឌ្ឍន៍គេហទំព័រ។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីគោលគំនិតមូលដ្ឋាននៃ PHP រួមមានអថេរ ប្រភេទទិន្នន័យ ថេរ ប្រមាណវិធី ស្ថាបត្យកម្មគ្រប់គ្រង និងរង្វិលជុំ។

2.1. អថេរ និងប្រភេទទិន្នន័យ

នៅក្នុង PHP អថេរត្រូវបានប្រើដើម្បីផ្ទុកទិន្នន័យ។ ពួកវាចាប់ផ្តើមដោយសញ្ញាដុល្លារ (`\$`) ហើយបន្ទាប់ដោយឈ្មោះអថេរ។ PHP គាំទ្រប្រភេទទិន្នន័យផ្សេងៗ រួមមានឃ្លា (string) ចំនួនគត់ (integer) ចំនួនទសភាគ (float) តម្លៃប្លូលីន (boolean) អារេ (array) និងវត្ថុ (object)។

```
// Example: Variables and Data Types
$name = "John Doe"; // String
$age = 25; // Integer
$height = 5.9; // Float
```

```
$isStudent = true; // Boolean
$fruits = array("Apple", "Banana", "Orange"); // Array

echo "Name: $name, Age: $age, Height: $height";
```

លទ្ធផល៖

```
Name: John Doe, Age: 25, Height: 5.9
```

2.2. ថេរ

ថេរគឺដូចអថេរ ប៉ុន្តែតម្លៃរបស់វាមិនអាចផ្លាស់ប្តូរបាននៅពេលដែលបានកំណត់។ ពួកវាត្រូវបានកំណត់ដោយប្រើអនុគមន៍ `define()` ឬពាក្យគន្លឹះ `const`។

```
// Example: Constants
define("PI", 3.14159);
const SITE_NAME = "MyWebsite";

echo "PI: " . PI . ", Site Name: " . SITE_NAME;
```

លទ្ធផល៖

```
PI: 3.14159, Site Name: MyWebsite
```

2.3. ប្រមាណវិធីនៅក្នុង PHP

PHP គាំទ្រប្រមាណវិធីផ្សេងៗ រួមមានប្រមាណវិធីគណនានិមិត្ត ប្រមាណវិធីប្រៀបធៀប ប្រមាណវិធីតភ្ជាប់ និងប្រមាណវិធីផ្សេងៗទៀត។

ប្រមាណវិធីគណនានិមិត្ត (Arithmetic Operators)

ប្រមាណវិធីគណនានិមិត្តត្រូវបានប្រើសម្រាប់ធ្វើប្រតិបត្តិការគណនាមូលដ្ឋានដូចជា បូក ដក គុណ និងចែក។

```
// Example: Arithmetic Operators
$a = 10;
$b = 5;

$sum = $a + $b; // Addition
$diff = $a - $b; // Subtraction
$product = $a * $b; // Multiplication
```

```
$quotient = $a / $b; // Division
$remainder = $a % $b; // Modulus (Remainder)

echo "Sum: $sum, Difference: $diff, Product: $product, Quotient: $quotient, Remainder: $remainder";
```

លទ្ធផល៖

```
Sum: 15, Difference: 5, Product: 50, Quotient: 2, Remainder: 0
```

ប្រមាណវិធីប្រៀបធៀប (Comparison Operators)

ប្រមាណវិធីប្រៀបធៀបត្រូវបានប្រើដើម្បីប្រៀបធៀបតម្លៃពីរ ហើយប្រគល់តម្លៃជា `true` ឬ `false`។

```
// Example: Comparison Operators
$a = 10;
$b = 5;

$isEqual = ($a == $b); // Equal to
$isNotEqual = ($a != $b); // Not equal to
$isGreater = ($a > $b); // Greater than
$isLess = ($a < $b); // Less than
$isGreaterOrEqual = ($a >= $b); // Greater than or equal to
$isLessOrEqual = ($a <= $b); // Less than or equal to

echo "Is Equal: $isEqual, Is Not Equal: $isNotEqual, Is Greater: $isGreater, Is Less: $isLess, Is Greater or Equal: $isGreaterOrEqual, Is Less or Equal: $isLessOrEqual";
```

លទ្ធផល៖

```
Is Equal: , Is Not Equal: 1, Is Greater: 1, Is Less: , Is Greater or Equal: 1, Is Less or Equal: 0
```

ប្រមាណវិធីតក្ក (Logical Operators)

ប្រមាណវិធីតក្កត្រូវបានប្រើដើម្បីផ្សំលក្ខខណ្ឌជាច្រើន។

```
// Example: Logical Operators
$a = true;
$b = false;

$and = ($a && $b); // Logical AND
```

```
$or = ($a || $b); // Logical OR
$not = !$a; // Logical NOT

echo "AND: $and, OR: $or, NOT: $not";
```

លទ្ធផល៖

AND: , OR: 1, NOT:

ប្រមាណវិធីផ្សំ (Assignment Operators)

ប្រមាណវិធីផ្សំត្រូវបានប្រើដើម្បីផ្តល់តម្លៃដល់អថេរ។

```
// Example: Assignment Operators
$a = 10;
$a += 5; // $a = $a + 5
$a -= 3; // $a = $a - 3
$a *= 2; // $a = $a * 2
$a /= 4; // $a = $a / 4

echo "Value of a: $a";
```

លទ្ធផល៖

Value of a: 6

2.4. ស្ថាបត្យកម្មគ្រប់គ្រង (if, if-else, if-elseif-else)

ស្ថាបត្យកម្មគ្រប់គ្រងអនុញ្ញាតឱ្យអ្នកគ្រប់គ្រងលំហូរនៃកម្មវិធីដោយផ្អែកលើលក្ខខណ្ឌ។ PHP គាំទ្រស្ថាបត្យកម្មដូចជា `if`, `if-else`, និង `if-elseif-else`។

if Statement

ស្ថាបត្យកម្ម `if` ត្រូវបានប្រើដើម្បីអនុវត្តកូដប្រសិនបើលក្ខខណ្ឌជាការពិត។

```
// Example: if Statement
$age = 20;

if ($age >= 18) {
```

```
    echo "You are an adult.";
}
```

លទ្ធផល៖

```
You are an adult.
```

if-else Statement

ស្ថាបត្យកម្ម `if-else` ត្រូវបានប្រើដើម្បីអនុវត្តកូដមួយប្រសិនបើលក្ខខណ្ឌជាការពិត និងកូដមួយទៀតប្រសិនបើលក្ខខណ្ឌមិនពិត។

```
// Example: if-else Statement
$age = 15;

if ($age >= 18) {
    echo "You are an adult.";
} else {
    echo "You are a minor.";
}
```

លទ្ធផល៖

```
You are a minor.
```

if-elseif-else Statement

ស្ថាបត្យកម្ម `if-elseif-else` ត្រូវបានប្រើដើម្បីពិនិត្យលក្ខខណ្ឌច្រើន។

```
// Example: if-elseif-else Statement
$score = 85;

if ($score >= 90) {
    echo "Grade: A";
} elseif ($score >= 80) {
    echo "Grade: B";
} elseif ($score >= 70) {
    echo "Grade: C";
} else {
```

```
    echo "Grade: D";  
}
```

លទ្ធផល៖

```
Grade: B
```

switch Statement Variations

ស្ថាបត្យកម្ម `switch` ក៏អាចប្រើសម្រាប់ពិនិត្យលក្ខខណ្ឌដូចគ្នានេះបានដែរ។ ខាងក្រោមជាឧទាហរណ៍ពីរបន្ថែម៖

```
// Switch Example 1: Using Conditions  
$score = 85;  
  
switch (true) {  
    case ($score >= 90):  
        echo "Grade: A";  
        break;  
    case ($score >= 80):  
        echo "Grade: B";  
        break;  
    case ($score >= 70):  
        echo "Grade: C";  
        break;  
    default:  
        echo "Grade: D";  
}
```

លទ្ធផល៖

```
Grade: B
```

```
// Switch Example 2: Using Score Ranges  
$score = 85;  
$gradeRange = (int)($score / 10);  
  
switch ($gradeRange) {  
    case 10:  
    case 9:  
        echo "Grade: A";  
}
```

```

        break;
    case 8:
        echo "Grade: B";
        break;
    case 7:
        echo "Grade: C";
        break;
    default:
        echo "Grade: D";
}

```

លទ្ធផល៖

Grade: B

2.5. រង្វិលជុំ (for, while, do-while, foreach)

រង្វិលជុំអនុញ្ញាតឱ្យអ្នកអនុវត្តកូដដដែលៗដោយផ្អែកលើលក្ខខណ្ឌ។ PHP គាំទ្ររង្វិលជុំដូចជា `for`, `while`, `do-while`, និង `foreach`។

for Loop

រង្វិលជុំ `for` ត្រូវបានប្រើដើម្បីអនុវត្តកូដច្រើនដង នៅពេលអ្នកដឹងចំនួនដងដែលអ្នកចង់អនុវត្ត។

```

// Example: for Loop
for ($i = 1; $i <= 5; $i++) {
    echo "Number: $i
";
}

```

លទ្ធផល៖

Number: 1
 Number: 2
 Number: 3
 Number: 4
 Number: 5

while Loop

រង្វិលជុំ `while` ត្រូវបានប្រើដើម្បីអនុវត្តកូដដដែលៗ ប្រសិនបើលក្ខខណ្ឌជាការពិត។

```
// Example: while Loop
$i = 1;

while ($i <= 5) {
    echo "Number: $i
";
    $i++;
}
```

លទ្ធផល៖

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

do-while Loop

រឿងលំដាប់ `do-while` ត្រូវបានប្រើដើម្បីអនុវត្តកូដយ៉ាងហោចណាស់មួយដង ហើយបន្តអនុវត្តប្រសិនបើលក្ខខណ្ឌជាការពិត។

```
// Example: do-while Loop
$i = 1;

do {
    echo "Number: $i
";
    $i++;
} while ($i <= 5);
```

លទ្ធផល៖

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

foreach Loop

រង្វិលជុំ `foreach` ត្រូវបានប្រើដើម្បីអនុវត្តកូដសម្រាប់ធាតុនីមួយៗក្នុងអារេ។

```
// Example: foreach Loop
$fruits = array("Apple", "Banana", "Orange");

foreach ($fruits as $fruit) {
    echo "$fruit
";
}
```

លទ្ធផល៖

```
Apple
Banana
Orange
```

3. Functions in PHP

អនុគមន៍នៅក្នុង PHP គឺជាផ្នែកមួយដ៏សំខាន់នៃការសរសេរកូដ។ ពួកវាអនុញ្ញាតឱ្យអ្នករៀបចំកូដជាប្លុកដែលអាចប្រើឡើងវិញបាន និងគ្រប់គ្រងលំហូរនៃកម្មវិធីយ៉ាងមានប្រសិទ្ធភាព។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីការកំណត់និងហៅអនុគមន៍ ជាពិសេសត្រូវតែចងចាំថា វិសាលភាពអថេរ និងអនុគមន៍ដែលមានស្រាប់នៅក្នុង PHP។

3.1. Defining and Calling Functions

អនុគមន៍នៅក្នុង PHP ត្រូវបានកំណត់ដោយប្រើពាក្យគន្លឹះ `function`។ អ្នកអាចហៅអនុគមន៍នេះនៅពេលណាក៏បាន ដើម្បីអនុវត្តកូដដែលបានកំណត់នៅក្នុងអនុគមន៍។

```
// ឧទាហរណ៍៖ កំណត់និងហៅអនុគមន៍
function greet() {
    echo "Hello, World!";
}

greet(); // ហៅអនុគមន៍
```

លទ្ធផល៖

```
Hello, World!
```

3.2. Function Parameters and Return Values

អនុគមន៍អាចទទួលបានការដាក់តម្លៃពីអ្នកប្រើប្រាស់ និងត្រឡប់តម្លៃដោយប្រើពាក្យគន្លឹះ `return`។

```
// ឧទាហរណ៍៖ ប្រើប្រាស់ប្រព័ន្ធគណិតសាស្ត្រ
function add($a, $b) {
    return $a + $b;
}

$result = add(5, 3); // តម្លៃត្រឡប់គឺ 8
echo "Result: $result";
```

លទ្ធផល៖

```
Result: 8
```

3.3. Variable Scope and Global Variables

អថេរដែលបានកំណត់នៅក្នុងអនុគមន៍មានវិសាលភាពក្នុងមុខងារនោះ។ ដើម្បីចូលប្រើអថេរក្រៅមុខងារ អ្នកអាចប្រើពាក្យគន្លឹះ `global`។

```
// ឧទាហរណ៍៖ វិសាលភាពអថេរ និងអថេរសកល
$x = 10; // អថេរសកល

function test() {
    global $x; // ចូលប្រើអថេរសកល
    echo "Value of x: $x";
}

test();
```

លទ្ធផល៖

```
Value of x: 10
```

3.4. Built-in PHP Functions

PHP មានអនុគមន៍ដែលមានស្រាប់ជាច្រើន ដែលអ្នកអាចប្រើដើម្បីអនុវត្តកិច្ចការផ្សេងៗ ដូចជាការធ្វើម៉ាស៊ីន គិតលេខ ការគ្រប់គ្រងឃ្លា និងការងារផ្សេងៗទៀត។

```
// ឧទាហរណ៍៖ អនុគមន៍ដែលមានស្រាប់
$string = "Hello, World!";
echo "Length: " . strlen($string); // ប្រវែងឃ្លា
echo "Uppercase: " . strtoupper($string); // បំប្លែងអក្សរធំ
```

លទ្ធផល៖

```
Length: 13
Uppercase: HELLO, WORLD!
```

4. Arrays in PHP

អារេ (Arrays) នៅក្នុង PHP គឺជាប្រភេទទិន្នន័យដែលអនុញ្ញាតឱ្យអ្នកផ្ទុកតម្លៃច្រើនក្នុងអថេរតែមួយ។ អារេអាចមានប្រភេទផ្សេងៗ ដូចជា អារេដែលមានសន្ទស្សន៍ (Indexed Arrays) អារេដែលមានគោលគំនិត (Associative Arrays) និងអារេពហុវិមាត្រ (Multidimensional Arrays)។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីប្រភេទអារេទាំងនេះ ព្រមទាំងមុខងារដែលផ្តល់ដោយ PHP ដើម្បីគ្រប់គ្រងអារេ។

4.1. Indexed Arrays

អារេដែលមានសន្ទស្សន៍ (Indexed Arrays) គឺជាអារេដែលតម្លៃត្រូវបានចងក្រងជាមួយសន្ទស្សន៍លេខ។ សន្ទស្សន៍ចាប់ផ្តើមពីលេខ 0។

```
// ឧទាហរណ៍៖ អារេដែលមានសន្ទស្សន៍
$fruits = array("Apple", "Banana", "Orange");

echo "First fruit: " . $fruits[0]; // ទទួលបានតម្លៃដំបូង
```

លទ្ធផល៖

```
First fruit: Apple
```

4.2. Associative Arrays

អារេដែលមានគោលគំនិត (Associative Arrays) គឺជាអារេដែលតម្លៃត្រូវបានចងភ្ជាប់ជាមួយសន្ទស្សន៍ដែលមានឈ្មោះ (keys)។

```
// ឧទាហរណ៍៖ អារេដែលមានគោលគំនិត
$person = array("name" => "John", "age" => 25, "city" => "Phnom Penh");

echo "Name: " . $person["name"]; // ទទួលបានតម្លៃដោយប្រើ key
```

លទ្ធផល៖

```
Name: John
```

4.3. Multidimensional Arrays

អារេពហុវិមាត្រ (Multidimensional Arrays) គឺជាអារេដែលមានអារេផ្សេងៗនៅក្នុងវា។ វាអនុញ្ញាតឱ្យអ្នកផ្គុំទិន្នន័យក្នុងទម្រង់តារាង។

```
// ឧទាហរណ៍៖ អារេពហុវិមាត្រ
$students = array(
    array("name" => "John", "age" => 20),
    array("name" => "Jane", "age" => 22),
    array("name" => "Doe", "age" => 21)
);

echo "First student name: " . $students[0]["name"]; // ទទួលបានតម្លៃពីអារេ
```

លទ្ធផល៖

```
First student name: John
```

4.4. Array Functions

PHP ផ្តល់មុខងារជាច្រើនដើម្បីគ្រប់គ្រងអារេ។ មុខងារទាំងនេះអនុញ្ញាតឱ្យអ្នកធ្វើការជាមួយអារេយ៉ាងមានប្រសិទ្ធភាព ដូចជាការបញ្ចូល ការលុប ការស្វែងរក និងការរៀបចំអារេ។

```
// ឧទាហរណ៍៖ មុខងារអារេ
$fruits = array("Apple", "Banana", "Orange");

// រាប់ចំនួនធាតុក្នុងអារេ
echo "Total fruits: " . count($fruits);

// បញ្ចូលធាតុថ្មីទៅអារេ
array_push($fruits, "Mango");
print_r($fruits);
```

លទ្ធផល៖

```
Total fruits: 3
Array
(
    [0] => Apple
    [1] => Banana
    [2] => Orange
    [3] => Mango
)
```

5. Strings in PHP

ឃ្លា (Strings) នៅក្នុង PHP គឺជាប្រភេទទិន្នន័យដែលផ្ទុកអក្សរ ឬអត្ថបទ។ PHP ផ្តល់មុខងារជាច្រើនដើម្បីធ្វើការជាមួយឃ្លា ដូចជាការបំបែក ការភ្ជាប់ ការស្វែងរក និងការជំនួស។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីមុខងារដែលពាក់ព័ន្ធនឹងឃ្លា ការភ្ជាប់ឃ្លា និងការប្រើកន្សោមធម្មតា (Regular Expressions)។

5.1. String Manipulation Functions

PHP ផ្តល់មុខងារជាច្រើនដើម្បីធ្វើការជាមួយឃ្លា។ មុខងារទាំងនេះអនុញ្ញាតឱ្យអ្នកធ្វើការជាមួយឃ្លាយ៉ាងមានប្រសិទ្ធភាព ដូចជាការបំបែក ការស្វែងរក និងការផ្លាស់ប្តូរឃ្លា។

```
// ឧទាហរណ៍: មុខងារធ្វើការជាមួយអ្នក
$text = "Hello, World!";

// ប្រវែងអ្នក
echo "Length: " . strlen($text);

// បំប្លែងអក្សរតូច
echo "Lowercase: " . strtolower($text);

// បំប្លែងអក្សរធំ
echo "Uppercase: " . strtoupper($text);
```

លទ្ធផល៖

```
Length: 13
Lowercase: hello, world!
Uppercase: HELLO, WORLD!
```

5.2. String Concatenation

ការភ្ជាប់អ្នក (String Concatenation) គឺជាការភ្ជាប់អ្នកពីរ ឬច្រើនជាមួយគ្នា។ នៅក្នុង PHP អ្នកអាចប្រើសញ្ញា `.` ដើម្បីភ្ជាប់អ្នក។

```
// ឧទាហរណ៍: ការភ្ជាប់អ្នក
$firstName = "John";
$lastName = "Doe";

// ភ្ជាប់អ្នក
$fullName = $firstName . " " . $lastName;
echo "Full Name: " . $fullName;
```

លទ្ធផល៖

```
Full Name: John Doe
```

5.3. Regular Expressions in PHP

កន្សោមធម្មតា (Regular Expressions) គឺជាវិធីដ៏មានឥទ្ធិពលដើម្បីស្វែងរក ឬផ្លាស់ប្តូរឃ្លាដោយប្រើលំនាំ។ PHP គាំទ្រកន្សោមធម្មតាតាមរយៈមុខងារ `preg\_match` និង `preg\_replace`។

```
// ឧទាហរណ៍៖ កន្សោមធម្មតា
$text = "The quick brown fox jumps over the lazy dog.";

// ស្វែងរកពាក្យ "fox"
if (preg_match("/fox/", $text)) {
    echo "Found 'fox' in the text.";
}

// ជំនួសពាក្យ "dog" ជាមួយ "cat"
$newText = preg_replace("/dog/", "cat", $text);
echo "New Text: " . $newText;
```

លទ្ធផល៖

```
Found 'fox' in the text.
New Text: The quick brown fox jumps over the lazy cat.
```

6. Superglobals in PHP

Superglobals គឺជាអថេរពិសេសនៅក្នុង PHP ដែលមានភាពងាយស្រួលក្នុងការចូលប្រើពីគ្រប់ទីកន្លែងនៅក្នុងស្ត្រីប។ ពួកវាត្រូវបានប្រើដើម្បីចូលប្រើទិន្នន័យពីទម្រង់ (forms) កុហក (cookies) និងព័ត៌មានអំពីស៊ីរី។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពី Superglobals ដូចជា `\$\_GET`, `\$\_POST`, `\$\_REQUEST`, `\$\_SERVER`, `\$\_SESSION`, និង `\$\_COOKIE`។

6.1. \$_GET and \$_POST

`\$\_GET` និង `\$\_POST` គឺជា Superglobals ដែលត្រូវបានប្រើដើម្បីចូលប្រើទិន្នន័យដែលបានផ្ញើតាមរយៈទម្រង់ HTML ដោយប្រើវិធីសាស្ត្រ GET ឬ POST។

```
// ឧទាហរណ៍៖ $_GET
// URL: http://example.com?name=John&age=25
echo "Name: " . $_GET['name']; // John
echo "Age: " . $_GET['age']; // 25
```

```
// ឧទាហរណ៍៖ $_POST
// ទម្រង់ HTML ដែលប្រើ method="POST"
echo "Name: " . $_POST['name'];
echo "Age: " . $_POST['age'];
```

លទ្ធផល៖

```
Name: John
Age: 25
```

6.2. \$_REQUEST

`\$\_REQUEST` គឺជា Superglobal ដែលផ្គុំទិន្នន័យដែលបានផ្ញើតាមរយៈទម្រង់ HTML ដោយប្រើវិធីសាស្ត្រ GET, POST, ឬ COOKIE។ វាជាការរួមបញ្ចូលនៃ `\$\_GET`, `\$\_POST`, និង `\$\_COOKIE`។

```
// ឧទាហរណ៍៖ $_REQUEST
// URL: http://example.com?name=John
echo "Name: " . $_REQUEST['name']; // John
```

លទ្ធផល៖

```
Name: John
```

6.3. \$_SERVER

`\$\_SERVER` គឺជា Superglobal ដែលផ្គុំព័ត៌មានអំពីស៊ីរ៉េ និងបរិស្ថាននៃកម្មវិធី។ វាផ្តល់ព័ត៌មានដូចជា URL បច្ចុប្បន្ន វិធីសាស្ត្រស្នើសុំ (request method) និងព័ត៌មានផ្សេងៗទៀត។

```
// ឧទាហរណ៍៖ $_SERVER
echo "Server Name: " . $_SERVER['SERVER_NAME']; // example.com
echo "Request Method: " . $_SERVER['REQUEST_METHOD']; // GET or POST
echo "Script Name: " . $_SERVER['SCRIPT_NAME']; // /index.php
```

លទ្ធផល៖

```
Server Name: example.com
Request Method: GET
```

6.4. \$_SESSION and \$_COOKIE

`\$\_SESSION` និង `\$\_COOKIE` គឺជា Superglobals ដែលត្រូវបានប្រើដើម្បីគ្រប់គ្រងសម័យ (sessions) និងកុហក (cookies)។ `\$\_SESSION` ផ្ទុកទិន្នន័យដែលត្រូវបានរក្សាទុកនៅលើស៊ីរីវ ខណៈពេលដែល `\$\_COOKIE` ផ្ទុកទិន្នន័យដែលត្រូវបានរក្សាទុកនៅលើកុំព្យូទ័រអ្នកប្រើប្រាស់។

```
// ឧទាហរណ៍: $_SESSION
session_start();
$_SESSION['username'] = 'JohnDoe';
echo "Session Username: " . $_SESSION['username']; // JohnDoe

// ឧទាហរណ៍: $_COOKIE
setcookie("user", "JohnDoe", time() + 3600, "/");
echo "Cookie User: " . $_COOKIE['user']; // JohnDoe
```

លទ្ធផល៖

```
Session Username: JohnDoe
Cookie User: JohnDoe
```

7. សំណុំបែបបទ និងការបញ្ចូលពីអ្នកប្រើ

សំណុំបែបបទ (Forms) គឺជាផ្នែកមួយដ៏សំខាន់នៃការអភិវឌ្ឍន៍គេហទំព័រ។ ពួកវាអនុញ្ញាតឱ្យអ្នកប្រើប្រាស់បញ្ចូលទិន្នន័យ ហើយទិន្នន័យនេះត្រូវបានផ្ញើទៅស៊ីរីវដើម្បីដោះស្រាយ។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីការគ្រប់គ្រងទិន្នន័យពីសំណុំបែបបទ ការធ្វើសុពលភាពទិន្នន័យ និងការការពារការរាយប្រហារដូចជា SQL Injection និង XSS។

7.1. ការគ្រប់គ្រងទិន្នន័យពីសំណុំបែបបទ

ទិន្នន័យពីសំណុំបែបបទអាចត្រូវបានផ្ញើតាមរយៈវិធីសាស្ត្រ GET ឬ POST។ PHP ផ្តល់ Superglobals ដូចជា `\$\_GET` និង `\$\_POST` ដើម្បីចូលប្រើទិន្នន័យនេះ។

```
// ឧទាហរណ៍៖ ការគ្រប់គ្រងទិន្នន័យពីសំណុំបែបបទ
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];
    $email = $_POST['email'];
    echo "Name: $name, Email: $email";
}
```

លទ្ធផល៖

Name: John Doe, Email: john@example.com

7.2. ការធ្វើសុពលភាពទិន្នន័យពីសំណុំបែបបទ

ការធ្វើសុពលភាពទិន្នន័យគឺជាដំណើរការនៃការធានាថាទិន្នន័យដែលបានបញ្ចូលដោយអ្នកប្រើប្រាស់គឺត្រឹមត្រូវ និងសុវត្ថិភាព។ នេះអាចរួមបញ្ចូលការពិនិត្យថាតើអ្នកប្រើប្រាស់បានបញ្ចូលអ្វីមែនត្រឹមត្រូវ ឬថាតើវាអាចបំពានច្បាប់បានបំពេញ។

```
// ឧទាហរណ៍៖ ការធ្វើសុពលភាពទិន្នន័យ
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];
    $email = $_POST['email'];

    if (empty($name)) {
        echo "Name is required.";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        echo "Invalid email format.";
    } else {
        echo "Form submitted successfully!";
    }
}
```

លទ្ធផល៖

Form submitted successfully!

7.3. ការការពារ SQL Injection និង XSS

SQL Injection និង XSS (Cross-Site Scripting) គឺជាការរាយប្រហារសុវត្ថិភាពដែលអាចបង្កឱ្យមានការខូចខាតធ្ងន់ធ្ងរដល់កម្មវិធីរបស់អ្នក។ ដើម្បីការពារការរាយប្រហារទាំងនេះ អ្នកគួរតែប្រើមុខងារដូចជា `mysqli\_real\_escape\_string` ឬ `htmlspecialchars`។

```
// ឧទាហរណ៍៖ ការការពារ SQL Injection
$name = mysqli_real_escape_string($conn, $_POST['name']);
$email = mysqli_real_escape_string($conn, $_POST['email']);

// ឧទាហរណ៍៖ ការការពារ XSS
$name = htmlspecialchars($_POST['name']);
$email = htmlspecialchars($_POST['email']);
```

លទ្ធផល៖

ទិន្នន័យត្រូវបានការពារពី SQL Injection និង XSS។

8. ការគ្រប់គ្រងកំហុស និងការបំបាត់កំហុស

ការគ្រប់គ្រងកំហុស និងការបំបាត់កំហុសគឺជាជំនាញសំខាន់សម្រាប់អ្នកអភិវឌ្ឍ JavaScript។ នៅក្នុងផ្នែកនេះ យើងនឹងស្វែងយល់ពីវិធីសាស្ត្រ console, ការបំបាត់កំហុសដោយប្រើ DevTools របស់កម្មវិធីរុករក និងការប្រើ `try...catch` និង `throw`។

8.1. វិធីសាស្ត្រ Console (console.log, console.error)

Console methods ដូចជា `console.log` និង `console.error` ត្រូវបានប្រើដើម្បីបង្ហាញព័ត៌មាននៅក្នុង console របស់កម្មវិធីរុករក។

```
// ឧទាហរណ៍៖ Console Methods
console.log("This is a log message."); // បង្ហាញសារធម្មតា
console.error("This is an error message."); // បង្ហាញសារកំហុស
```

លទ្ធផល៖

This is a log message.
This is an error message.

8.2. ការបំបាត់កំហុសដោយប្រើ DevTools

DevTools របស់កម្មវិធីរុករកផ្តល់ឧបករណ៍សម្រាប់បំបាត់កំហុស ដូចជា breakpoints, step-through debugging, និងការតាមដានតម្លៃអថេរ។

```
// ឧទាហរណ៍៖ Debugging with DevTools
function add(a, b) {
  debugger; // Pause execution here
  return a + b;
}

console.log(add(2, 3));
```

លទ្ធផល៖

នៅពេលដំណើរការ កូដនឹងផ្អាកនៅ `debugger` ហើយអ្នកអាចពិនិត្យតម្លៃអថេរនៅក្នុង DevTools បាន។

8.3. try...catch និង throw

`try...catch` អនុញ្ញាតឱ្យអ្នកគ្រប់គ្រងកំហុសដោយមិនធ្វើឱ្យកម្មវិធីរបស់អ្នកឈប់ដំណើរការ។ `throw` អនុញ្ញាតឱ្យអ្នកបង្កើតកំហុសផ្ទាល់ខ្លួន។

```
// ឧទាហរណ៍៖ try...catch និង throw
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero!");
  }
  return a / b;
}

try {
  console.log(divide(10, 0));
} catch (error) {
  console.error(error.message); // បង្ហាញសារកំហុស
}
```

លទ្ធផល៖

Cannot divide by zero!

8. Working with Files in PHP

PHP ផ្តល់មុខងារជាច្រើនដើម្បីធ្វើការជាមួយឯកសារ ដូចជាការអាន ការសរសេរ ការផ្ទុកឡើង និងការគ្រប់គ្រងឯកសារនៅលើស៊ីរ៉ែ។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីការគ្រប់គ្រងឯកសារ ការផ្ទុកឡើងឯកសារ និងការធ្វើការជាមួយឯកសារប្រភេទ CSV និង JSON។

8.1. File Handling (Read/Write)

PHP អនុញ្ញាតឱ្យអ្នកអាន និងសរសេរទិន្នន័យទៅកាន់ឯកសារ។ អ្នកអាចប្រើមុខងារដូចជា `fopen`, `fread`, `fwrite`, និង `fclose` ដើម្បីធ្វើការជាមួយឯកសារ។

```
// ឧទាហរណ៍: ការសរសេរទិន្នន័យទៅកាន់ឯកសារ
$file = fopen("example.txt", "w");
fwrite($file, "Hello, World!");
fclose($file);

// ឧទាហរណ៍: ការអានទិន្នន័យពីឯកសារ
$file = fopen("example.txt", "r");
echo fread($file, filesize("example.txt"));
fclose($file);
```

លទ្ធផល:

Hello, World!

8.2. Uploading Files

PHP អនុញ្ញាតឱ្យអ្នកផ្ទុកឡើងឯកសារពីទម្រង់ HTML ទៅកាន់ស៊ីរ៉ែ។ អ្នកអាចប្រើ Superglobal `\$\_FILES` ដើម្បីចូលប្រើឯកសារដែលបានផ្ទុកឡើង។

```
// ឧទាហរណ៍: ការផ្ទុកឡើងឯកសារ
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $target_dir = "uploads/";
```

```

    $target_file = $target_dir . basename($_FILES["fileToUpload"]["name"]);
    move_uploaded_file($_FILES["fileToUpload"]["tmp_name"], $target_file);
    echo "File uploaded successfully.";
}

```

លទ្ធផល៖

```
File uploaded successfully.
```

8.3. Handling CSV and JSON Files

PHP អនុញ្ញាតឱ្យអ្នកធ្វើការជាមួយឯកសារប្រភេទ CSV និង JSON។ អ្នកអាចប្រើមុខងារដូចជា `fgetcsv` ដើម្បីអានឯកសារ CSV និង `json\_encode` និង `json\_decode` ដើម្បីធ្វើការជាមួយទិន្នន័យ JSON។

```

// ឧទាហរណ៍៖ ការអានឯកសារ CSV
$file = fopen("example.csv", "r");
while (($data = fgetcsv($file)) !== FALSE) {
    print_r($data);
}
fclose($file);

// ឧទាហរណ៍៖ ការធ្វើការជាមួយទិន្នន័យ JSON
$data = array("name" => "John", "age" => 25);
$json = json_encode($data);
echo $json;

$decoded = json_decode($json, true);
print_r($decoded);

```

លទ្ធផល៖

```

Array
(
    [0] => John
    [1] => 25
)
{"name":"John","age":25}
Array
(

```

```
[name] => John  
[age] => 25  
)
```

9. Object-Oriented Programming (OOP) in PHP

ការសរសេរកម្មវិធីដោយប្រើប្រាស់វិធីសាស្ត្រប្រធានវត្ថុ (Object-Oriented Programming - OOP) គឺជាវិធីសាស្ត្រដ៏មានប្រសិទ្ធភាពក្នុងការរៀបចំកូដ។ PHP គាំទ្រវិធីសាស្ត្រនេះ ដោយផ្តល់លក្ខណៈពិសេសដូចជា Classes, Objects, Inheritance, Polymorphism, Interfaces, Abstract Classes, និង Traits។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីគោលគំនិតទាំងនេះ។

9.1. Classes and Objects

Class គឺជាគំរូ (blueprint) សម្រាប់បង្កើត Objects។ Object គឺជាករណីជាក់លាក់នៃ Class។ នៅក្នុង PHP អ្នកអាចកំណត់ Class ដោយប្រើពាក្យគន្លឹះ `class` ហើយបង្កើត Object ដោយប្រើពាក្យគន្លឹះ `new`។

```
// ឧទាហរណ៍៖ Class និង Object  
class Car {  
    public $brand;  
    public $model;  
  
    public function displayInfo() {  
        return "Brand: " . $this->brand . ", Model: " . $this->model;  
    }  
}  
  
$myCar = new Car();  
$myCar->brand = "Toyota";  
$myCar->model = "Corolla";  
echo $myCar->displayInfo();
```

លទ្ធផល៖

```
Brand: Toyota, Model: Corolla
```

9.2. Constructors and Destructors

Constructor គឺជាមុខងារពិសេសដែលត្រូវបានហៅដោយស្វ័យប្រវត្តិនៅពេល Object ត្រូវបានបង្កើត។
Destructor គឺជាមុខងារពិសេសដែលត្រូវបានហៅដោយស្វ័យប្រវត្តិនៅពេល Object ត្រូវបានបំផ្លាញ។

```
// ឧទាហរណ៍៖ Constructor និង Destructor
class Car {
    public $brand;
    public $model;

    public function __construct($brand, $model) {
        $this->brand = $brand;
        $this->model = $model;
        echo "Car object created.<br>";
    }

    public function __destruct() {
        echo "Car object destroyed.<br>";
    }

    public function displayInfo() {
        return "Brand: " . $this->brand . ", Model: " . $this->model;
    }
}

$myCar = new Car("Toyota", "Corolla");
echo $myCar->displayInfo();
```

លទ្ធផល៖

```
Car object created.
Brand: Toyota, Model: Corolla
Car object destroyed.
```

9.3. Inheritance and Polymorphism

Inheritance គឺជាលក្ខណៈពិសេសនៃ OOP ដែលអនុញ្ញាតឱ្យ Class មួយទទួលមរតកលក្ខណៈពិសេសពី Class ផ្សេងៗ។ Polymorphism គឺជាលក្ខណៈពិសេសដែលអនុញ្ញាតឱ្យមានច្រើនទម្រង់នៃមុខងារដូចគ្នា។

```
// ឧទាហរណ៍៖ Inheritance និង Polymorphism
class Vehicle {
    public function startEngine() {
        return "Engine started.<br>";
    }
}

class Car extends Vehicle {
    public function startEngine() {
        return "Car engine started.<br>";
    }
}

$vehicle = new Vehicle();
$car = new Car();
echo $vehicle->startEngine();
echo $car->startEngine();
```

លទ្ធផល៖

```
Engine started.
Car engine started.
```

9.4. Interfaces and Abstract Classes

Interface គឺជាការកំណត់រចនាសម្ព័ន្ធសម្រាប់ Class ដែលត្រូវតាម។ Abstract Class គឺជា Class ដែលមិនអាចបង្កើត Object ដោយផ្ទាល់ ហើយត្រូវតែត្រូវបានពង្រីកដោយ Class ផ្សេងៗ។

```
// ឧទាហរណ៍៖ Interface និង Abstract Class
interface Drivable {
    public function drive();
}

abstract class Vehicle {
    abstract public function startEngine();
}

class Car extends Vehicle implements Drivable {
    public function startEngine() {
```

```

        return "Car engine started.<br>";
    }

    public function drive() {
        return "Car is driving.<br>";
    }
}

$car = new Car();
echo $car->startEngine();
echo $car->drive();

```

លទ្ធផល៖

```

Car engine started.
Car is driving.

```

9.5. Traits in PHP

Traits គឺជាវិធីសាស្ត្រដើម្បីចែករំលែកមុខងាររវាង Classes។ Traits អនុញ្ញាតឱ្យអ្នកប្រើមុខងារដូចគ្នានៅក្នុង Classes ច្រើនដោយមិនចាំបាច់ពង្រីក Class មួយ។

```

// ឧទាហរណ៍៖ Traits
trait Loggable {
    public function log($message) {
        echo "Log: " . $message . "<br>";
    }
}

class Car {
    use Loggable;

    public function startEngine() {
        $this->log("Car engine started.");
        return "Engine started.<br>";
    }
}

```

```
$car = new Car();  
echo $car->startEngine();
```

លទ្ធផល៖

```
Log: Car engine started.  
Engine started.
```

10. PHP and MySQL

PHP និង MySQL គឺជាបច្ចេកវិទ្យាដ៏ពេញនិយមសម្រាប់ការអភិវឌ្ឍន៍កម្មវិធីគេហទំព័រដែលមានការភ្ជាប់ទៅនឹងមូលដ្ឋានទិន្នន័យ។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីការភ្ជាប់ទៅមូលដ្ឋានទិន្នន័យ MySQL ការអនុវត្តប្រតិបត្តិការ CRUD (Create, Read, Update, Delete) ការប្រើ Prepared Statements ដើម្បីការពារសុវត្ថិភាព និងការប្រើ PDO សម្រាប់ការទំនាក់ទំនងជាមួយមូលដ្ឋានទិន្នន័យ។

10.1. Connecting to a Database

ដើម្បីភ្ជាប់ទៅមូលដ្ឋានទិន្នន័យ MySQL អ្នកអាចប្រើមុខងារ `mysqli\_connect` ឬ PDO។ ខាងក្រោមនេះគឺជាឧទាហរណ៍នៃការភ្ជាប់ទៅមូលដ្ឋានទិន្នន័យ។

```
// ឧទាហរណ៍៖ ការភ្ជាប់ទៅមូលដ្ឋានទិន្នន័យ MySQL  
$servername = "localhost";  
$username = "root";  
$password = "";  
$dbname = "myDB";  
  
// បង្កើតការភ្ជាប់  
$conn = mysqli_connect($servername, $username, $password, $dbname);  
  
// ពិនិត្យការភ្ជាប់  
if (!$conn) {  
    die("Connection failed: " . mysqli_connect_error());  
}  
echo "Connected successfully";
```

លទ្ធផល៖

Connected successfully

10.2. CRUD Operations (Create, Read, Update, Delete)

CRUD គឺជាប្រតិបត្តិការមូលដ្ឋានដែលត្រូវបានប្រើក្នុងការគ្រប់គ្រងទិន្នន័យ។ ខាងក្រោមនេះគឺជាឧទាហរណ៍នៃប្រតិបត្តិការ CRUD។

```
// ឧទាហរណ៍៖ CRUD Operations
// Create
$sql = "INSERT INTO users (name, email) VALUES ('John Doe', 'john@example.com')";
if (mysqli_query($conn, $sql)) {
    echo "New record created successfully";
}

// Read
$sql = "SELECT id, name, email FROM users";
$result = mysqli_query($conn, $sql);
while ($row = mysqli_fetch_assoc($result)) {
    echo "ID: " . $row["id"] . " - Name: " . $row["name"] . " - Email: " . $row["email"] . "\n";
}

// Update
$sql = "UPDATE users SET email='john.doe@example.com' WHERE id=1";
if (mysqli_query($conn, $sql)) {
    echo "Record updated successfully";
}

// Delete
$sql = "DELETE FROM users WHERE id=1";
if (mysqli_query($conn, $sql)) {
    echo "Record deleted successfully";
}
```

លទ្ធផល៖

New record created successfully
ID: 1 - Name: John Doe - Email: john@example.com

```
Record updated successfully
Record deleted successfully
```

10.3. Prepared Statements and Security

Prepared Statements គឺជាវិធីសាស្ត្រដើម្បីការពារកម្មវិធីពីការវាយប្រហារដូចជា SQL Injection។ វាអនុញ្ញាតឱ្យអ្នកដាក់ប៉ារ៉ាម៉ែត្រទៅក្នុងសំណួរដោយសុវត្ថិភាព។

```
// ឧទាហរណ៍៖ Prepared Statements
$stmt = $conn->prepare("INSERT INTO users (name, email) VALUES (?, ?)");
$stmt->bind_param("ss", $name, $email);

// កំណត់តម្លៃប៉ារ៉ាម៉ែត្រ និងអនុវត្ត
$name = "Jane Doe";
$email = "jane@example.com";
$stmt->execute();

echo "New record created successfully";
```

លទ្ធផល៖

```
New record created successfully
```

10.4. Using PDO for Database Interaction

PDO (PHP Data Objects) គឺជាវិធីសាស្ត្រដ៏មានប្រសិទ្ធភាពសម្រាប់ការទំនាក់ទំនងជាមួយមូលដ្ឋានទិន្នន័យ។ វាផ្តល់ភាពអាចផ្លាស់ប្តូរបានរវាងមូលដ្ឋានទិន្នន័យផ្សេងៗ និងគាំទ្រការប្រើ Prepared Statements។

```
// ឧទាហរណ៍៖ ការប្រើ PDO
try {
    $conn = new PDO("mysql:host=$servername;dbname=$dbname", $username, $password);
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // ប្រើ Prepared Statements
    $stmt = $conn->prepare("INSERT INTO users (name, email) VALUES (:name, :email)");
    $stmt->bindParam(':name', $name);
    $stmt->bindParam(':email', $email);
```

```
// កំណត់តម្លៃប៉ារ៉ាម៉ែត្រ និងអនុវត្ត
$name = "Alice";
$email = "alice@example.com";
$stmt->execute();

echo "New record created successfully";
} catch (PDOException $e) {
    echo "Error: " . $e->getMessage();
}
```

លទ្ធផល៖

```
New record created successfully
```

11. Sessions and Cookies

Sessions និង Cookies គឺជាវិធីសាស្ត្រសម្រាប់គ្រប់គ្រងស្ថានភាពរបស់អ្នកប្រើប្រាស់នៅលើគេហទំព័រ។ Sessions ត្រូវបានរក្សាទុកនៅលើស៊ើវ ខណៈពេលដែល Cookies ត្រូវបានរក្សាទុកនៅលើកុំព្យូទ័រអ្នកប្រើប្រាស់។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីការគ្រប់គ្រង Sessions និងការប្រើ Cookies សម្រាប់ការតាមដានអ្នកប្រើប្រាស់។

11.1. Managing Sessions

Sessions ត្រូវបានប្រើដើម្បីរក្សាទិន្នន័យរបស់អ្នកប្រើប្រាស់នៅលើស៊ើវ ហើយអាចប្រើបាននៅទូទាំងទំព័រផ្សេងៗនៃគេហទំព័រ។ ដើម្បីប្រើ Sessions អ្នកត្រូវចាប់ផ្តើម Session ដោយប្រើមុខងារ `session\_start()`។

```
// ឧទាហរណ៍៖ ការគ្រប់គ្រង Sessions
session_start();

// កំណត់តម្លៃ Session
$_SESSION['username'] = 'JohnDoe';

// ទាញយកតម្លៃ Session
echo "Username: " . $_SESSION['username'];
```

```
// លុប Session
session_unset();
session_destroy();
```

លទ្ធផល៖

Username: JohnDoe

11.2. Using Cookies for User Tracking

Cookies ត្រូវបានប្រើដើម្បីរក្សាទិន្នន័យរបស់អ្នកប្រើប្រាស់នៅលើកុំព្យូទ័ររបស់ពួកគេ។ Cookies អាចត្រូវបានប្រើដើម្បីតាមដានអ្នកប្រើប្រាស់ ឬរក្សាការកំណត់របស់ពួកគេ។

```
// ឧទាហរណ៍៖ ការប្រើ Cookies
// កំណត់ Cookie
setcookie("username", "JohnDoe", time() + (86400 * 30), "/"); // 86400 =

// ទាញយកតម្លៃ Cookie
if (isset($_COOKIE['username'])) {
    echo "Username: " . $_COOKIE['username'];
} else {
    echo "Cookie not set.";
}

// លុប Cookie
setcookie("username", "", time() - 3600, "/");
```

លទ្ធផល៖

Username: JohnDoe

12. PHP and AJAX

AJAX (Asynchronous JavaScript and XML) គឺជាវិធីសាស្ត្រដែលអនុញ្ញាតឱ្យអ្នកធ្វើការស្នើសុំទៅកាន់ស៊ីរ៉េដោយមិនចាំបាច់ផ្អាកទំព័រឡើងវិញ។ PHP អាចត្រូវបានប្រើជាមួយ AJAX ដើម្បីធ្វើ

ការស្នើសុំអសមកាល និងទទួលការឆ្លើយតបពីស៊ីរ៉េ។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីការគ្រប់គ្រងសំណើអសមកាល និងការប្រើ Fetch API ជាមួយ PHP។

12.1. Handling Asynchronous Requests

AJAX អនុញ្ញាតឱ្យអ្នកធ្វើការស្នើសុំទៅកាន់ស៊ីរ៉េដោយមិនចាំបាច់ផ្ទុកទំព័រឡើងវិញ។ PHP អាចដំណើរការសំណើទាំងនេះ ហើយត្រឡប់ទិន្នន័យជាទម្រង់ JSON ឬ HTML។

```
// ឧទាហរណ៍៖ ការគ្រប់គ្រងសំណើអសមកាល
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];
    $response = array("status" => "success", "message" => "Hello, " . $name);
    echo json_encode($response);
}
```

លទ្ធផល៖

```
{"status":"success","message":"Hello, John"}
```

12.2. Fetch API and PHP

Fetch API គឺជាវិធីសាស្ត្រទំនើបសម្រាប់ធ្វើការស្នើសុំ HTTP ពី JavaScript។ វាអាចត្រូវបានប្រើជាមួយ PHP ដើម្បីទទួល និងដំណើរការទិន្នន័យពីស៊ីរ៉េ។

```
// ឧទាហរណ៍៖ ការប្រើ Fetch API ជាមួយ PHP
fetch('server.php', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({ name: 'John' })
})
.then(response => response.json())
.then(data => {
  console.log(data);
})
.catch(error => {
```

```
console.error('Error:', error);
});
```

លទ្ធផល៖

```
{"status": "success", "message": "Hello, John"}
```

13. PHP Security

សុវត្ថិភាពគឺជាផ្នែកមួយដ៏សំខាន់នៃការអភិវឌ្ឍន៍កម្មវិធី។ PHP ផ្តល់ឧបករណ៍និងវិធីសាស្ត្រជាច្រើនដើម្បីការពារកម្មវិធីពីការរាយប្រហារផ្សេងៗ។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីការធ្វើសុពលភាពនិងសម្អាតទិន្នន័យ ការការពារការរាយប្រហារ SQL Injection ការការពារការរាយប្រហារ Cross-Site Scripting (XSS) និងការគ្រប់គ្រងពាក្យសម្ងាត់ដោយសុវត្ថិភាព។

13.1. Input Validation and Sanitization

ការធ្វើសុពលភាពនិងសម្អាតទិន្នន័យគឺជាដំណើរការនៃការធានាថាទិន្នន័យដែលបានបញ្ចូលដោយអ្នកប្រើប្រាស់គឺត្រឹមត្រូវ និងសុវត្ថិភាព។ PHP ផ្តល់មុខងារដូចជា `filter\_var` និង `htmlspecialchars` សម្រាប់ការសម្អាតទិន្នន័យ។

```
// ឧទាហរណ៍៖ ការធ្វើសុពលភាពនិងសម្អាតទិន្នន័យ
$email = "john@example.com";

// ធ្វើសុពលភាពអ៊ីមែល
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Email is valid.";
} else {
    echo "Invalid email format.";
}

// សម្អាតទិន្នន័យ
$input = "<script>alert('XSS');</script>";
$cleanInput = htmlspecialchars($input, ENT_QUOTES, 'UTF-8');
echo $cleanInput;
```

លទ្ធផល៖

```
Email is valid.
```

```
<script>alert('#XSS#');</script>
```

13.2. Protecting Against SQL Injection

SQL Injection គឺជាការវាយប្រហារដែលអាចឱ្យអ្នកវាយប្រហារបញ្ចូលសំណើ SQL មិនសុវត្ថិភាពទៅក្នុងមូលដ្ឋានទិន្នន័យ។ ដើម្បីការពារការវាយប្រហារនេះ អ្នកគួរតែប្រើ Prepared Statements។

```
// ឧទាហរណ៍៖ ការការពារការវាយប្រហារ SQL Injection
$stmt = $conn->prepare("SELECT * FROM users WHERE email = ?");
$stmt->bind_param("s", $email);

$email = "john@example.com";
$stmt->execute();
$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
    echo "Name: " . $row['name'];
}
```

លទ្ធផល៖

```
Name: John Doe
```

13.3. Preventing Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) គឺជាការវាយប្រហារដែលអាចឱ្យអ្នកវាយប្រហារបញ្ចូលស្រ្តីបមិនសុវត្ថិភាពទៅក្នុងទំព័រ។ ដើម្បីការពារការវាយប្រហារនេះ អ្នកគួរតែប្រើមុខងារដូចជា `htmlspecialchars` ដើម្បីសម្អាតទិន្នន័យ។

```
// ឧទាហរណ៍៖ ការការពារ XSS
$input = "<script>alert('XSS');</script>";
$cleanInput = htmlspecialchars($input, ENT_QUOTES, 'UTF-8');
echo $cleanInput;
```

លទ្ធផល៖

```
<script>alert('#XSS#');</script>
```

13.4. Password Hashing and Authentication

ការគ្រប់គ្រងពាក្យសម្ងាត់ដោយសុវត្ថិភាពគឺជាផ្នែកមួយដ៏សំខាន់នៃការអភិវឌ្ឍន៍កម្មវិធី។ PHP ផ្តល់មុខងារដូចជា `password\_hash` និង `password\_verify` ដើម្បីគ្រប់គ្រងពាក្យសម្ងាត់ដោយសុវត្ថិភាព។

```
// ឧទាហរណ៍: ការគ្រប់គ្រងពាក្យសម្ងាត់
$password = "myPassword123";

// បង្កើត hash សម្រាប់ពាក្យសម្ងាត់
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);
echo "Hashed Password: " . $hashedPassword;

// ផ្ទៀងផ្ទាត់ពាក្យសម្ងាត់
if (password_verify($password, $hashedPassword)) {
    echo "Password is valid!";
} else {
    echo "Invalid password!";
}
```

លទ្ធផល៖

```
Hashed Password: $2y$10$examplehash
Password is valid!
```

14. Working with APIs in PHP

APIs (Application Programming Interfaces) គឺជាវិធីសាស្ត្រដែលអនុញ្ញាតឱ្យកម្មវិធីផ្សេងៗ ទំនាក់ទំនងជាមួយគ្នា។ PHP អាចត្រូវបានប្រើដើម្បីទទួលទិន្នន័យពី REST APIs ឬបង្កើត RESTful APIs ផ្ទាល់ខ្លួន។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពីការប្រើ REST APIs និងការបង្កើត RESTful APIs ជាមួយ PHP។

14.1. Consuming REST APIs

PHP អាចត្រូវបានប្រើដើម្បីទទួលទិន្នន័យពី REST APIs ដោយប្រើមុខងារដូចជា `file\_get\_contents` ឬ `cURL`។ ខាងក្រោមនេះគឺជាឧទាហរណ៍នៃការទទួលទិន្នន័យពី REST API។

```
// ឧទាហរណ៍: ការទទួលទិន្នន័យពី REST API
$url = "https://api.example.com/data";
$response = file_get_contents($url);
$data = json_decode($response, true);

print_r($data);
```

លទ្ធផល៖

```
Array
(
    [id] => 1
    [name] => John Doe
    [email] => john@example.com
)
```

14.2. Creating RESTful APIs with PHP

PHP អាចត្រូវបានប្រើដើម្បីបង្កើត RESTful APIs ដែលអនុញ្ញាតឱ្យកម្មវិធីផ្សេងៗទទួលទិន្នន័យពីស៊ីរ៉េរបស់អ្នក។ ខាងក្រោមនេះគឺជាឧទាហរណ៍នៃការបង្កើត RESTful API សាមញ្ញ។

```
// ឧទាហរណ៍: ការបង្កើត RESTful API
header("Content-Type: application/json");

$method = $_SERVER['REQUEST_METHOD'];
$request = explode("/", trim($_SERVER['PATH_INFO'], '/'));

switch ($method) {
    case 'GET':
        // ទទួលទិន្នន័យ
        echo json_encode(array("message" => "GET request received"));
        break;
    case 'POST':
        // បង្កើតទិន្នន័យ
        $data = json_decode(file_get_contents("php://input"), true);
        echo json_encode(array("message" => "POST request received", "data"
        break;
    default:
        // ការស្នើសុំមិនត្រឹមត្រូវ
```

```
http_response_code(405);  
echo json_encode(array("message" => "Method not allowed"));  
break;  
}
```

លទ្ធផល៖

```
{"message": "GET request received"}
```

15. PHP Frameworks

PHP Frameworks គឺជាឧបករណ៍ដែលផ្តល់នូវរចនាសម្ព័ន្ធ និងមុខងារសម្រាប់ការអភិវឌ្ឍន៍កម្មវិធី យ៉ាងមានប្រសិទ្ធភាព។ ក្នុងចំណោម PHP Frameworks ដែលពេញនិយមបំផុតគឺ Laravel។ នៅក្នុងផ្នែកនេះ យើងនឹងរៀនអំពី Laravel ដែលរួមមានការណែនាំអំពី Laravel, Routing និង Controllers, Blade Templates, និង Eloquent ORM។

15.1. Introduction to Laravel

Laravel គឺជា PHP Framework ដ៏ពេញនិយមមួយដែលផ្តល់នូវរចនាសម្ព័ន្ធសម្រាប់ការអភិវឌ្ឍន៍កម្មវិធីយ៉ាងមានប្រសិទ្ធភាព។ វាមានមុខងារជាច្រើនដូចជា Routing, Controllers, Blade Templates, និង Eloquent ORM។

```
# ការដំឡើង Laravel  
composer create-project --prefer-dist laravel/laravel blog
```

លទ្ធផល៖

```
Laravel project created successfully.
```

15.2. Routing and Controllers

Routing គឺជាវិធីសាស្ត្រដើម្បីកំណត់ពីរបៀបដែលកម្មវិធីឆ្លើយតបទៅនឹងសំណើ HTTP។ Controllers គឺជាផ្នែកមួយដែលគ្រប់គ្រងឡដ៏ទល់នៃកម្មវិធី។

```
// ឧទាហរណ៍៖ Routing និង Controllers
// កំណត់ Route
Route::get('/user', [UserController::class, 'index']);

// UserController
class UserController extends Controller
{
    public function index()
    {
        return "Hello, User!";
    }
}
```

លទ្ធផល៖

Hello, User!

15.3. Blade Templates

Blade គឺជាប្រព័ន្ធកំរូ (template engine) របស់ Laravel ដែលផ្តល់នូវភាពងាយស្រួលក្នុងការបង្កើតទំព័រ HTML។ Blade អនុញ្ញាតឱ្យអ្នកប្រើ syntax ដ៏សាមញ្ញ និងមានថាមពល។

```
// ឧទាហរណ៍៖ Blade Template
// resources/views/welcome.blade.php
<html>
<body>
    <h1>Hello, {{ $name }}</h1>
</body>
</html>

// Route
Route::get('/welcome', function () {
    return view('welcome', ['name' => 'John Doe']);
});
```

លទ្ធផល៖

```
<html>
<body>
```

```
<h1>Hello, John Doe</h1>
</body>
</html>
```

15.4. Eloquent ORM

Eloquent ORM គឺជាប្រព័ន្ធគ្រប់គ្រងទិន្នន័យរបស់ Laravel ដែលផ្តល់នូវវិធីសាស្ត្រដ៏សាមញ្ញ និងមានថាមពល សម្រាប់ការទំនាក់ទំនងជាមួយមូលដ្ឋានទិន្នន័យ។

```
// ឧទាហរណ៍៖ Eloquent ORM
// កំណត់ Model
class User extends Model
{
    protected $table = 'users';
}

// ទាញយកទិន្នន័យពីមូលដ្ឋានទិន្នន័យ
$users = User::all();
foreach ($users as $user) {
    echo $user->name;
}
```

លទ្ធផល៖

```
John Doe
Jane Doe
```

អំពីយើង

CodeKhmerLearning ផ្តល់ឱ្យអ្នកនូវគ្រឹះស្ថានភាពសិក្សាស្រាវជ្រាវក្នុងការរៀនកូដភាសាខ្មែរ។ យើងមានក្រុម ការងារព្រមមានជំនាញក្នុងការអភិវឌ្ឍកម្មវិធី និងវេបសាយ។

© 2025 CodeKhmerLearning. All Rights Reserved.

តំណភ្ជាប់រហ័ស

ផ្ទាំងដើម
វគ្គសិក្សា
អំពីយើង
ទំនាក់ទំនង

ទំនាក់ទំនង

សូមទំនាក់ទំនងមកកាន់យើងក្តី:

Email: thymuoyhak.biu@gmail.com

Phone: +855 888 052 812



© 2025 CodeKhmerLearning. All rights reserved.